Real Coder - Docker Instructions

Changelog

Date	Description
Feb 24, 2026	Creation of the doc

Docker Environment Setup & Execution Guide

This document explains how to correctly configure and run the Docker environment for your tasks in the Real Coder project.

IMPORTANT: It is extremely important that all instructions in this document are followed precisely. The validation script depends on a strict structure and any unintended changes may break the evaluation process.

Docker Environment

Requirements

1. **Use the provided Dockerfile template:** The provided Dockerfile template MUST be used, and can ONLY be modified for the installation of required dependencies for your solution and your tests.

   Make sure you don't:

   - Modify the structure of the file
   - Remove any existing instructions
   - Add unrelated layers of steps
   - Change entry points or default commands

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.225Z

```shell
##############################################
# BASE IMAGE
##############################################
FROM ubuntu:22.04


ENV DEBIAN_FRONTEND=noninteractive


##############################################
# SYSTEM DEPENDENCIES
##############################################
RUN apt-get update && apt-get install -y \
    git \
    python3 \
    python3-pip \
    python3-setuptools \
    python-is-python3 \
    unzip \
    && rm -rf /var/lib/apt/lists/*


##############################################
# WORKING DIRECTORY + GIT SETUP
##############################################
WORKDIR /app
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.225Z

```
RUN git init \
    && git config --global user.email "agent@example.com" \
    && git config --global user.name "Agent" \
    && echo "# Workspace" > README.md \
    && git add README.md \
    && git commit -m "Initial commit"

##############################################
# EVALUATION ASSETS DIRECTORY
##############################################
# Populated at RUNTIME by the evaluation script.
RUN mkdir -p /eval_assets

CMD ["/bin/bash"]
```

2. ***run.sh*** **file based on the template:** The run.sh file MUST be created based on the following template, so the validation can work as expected. Keep in mind that the file can only be changed within the delimited comments.

```Shell
#!/bin/bash
### COMMON SETUP; DO NOT MODIFY ###
set -e
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.225Z

```
# --- CONFIGURE THIS SECTION ---
# Replace this with your command to run all tests
run_all_tests() {
  echo "Running all tests..."
  # TODO: Run the full test suite
  # Example: cargo test --workspace --lib --no-fail-fast
}
# --- END CONFIGURATION SECTION ---


### COMMON EXECUTION; DO NOT MODIFY ###
run_all_tests
```

3. ***parsing.py*** **file based on the template:** The parsing.py file MUST also be based on the following template. Being edited and worked on only in the specified section.

```python
import dataclasses
import json
import sys
from enum import Enum
from pathlib import Path
from typing import List
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.225Z

```python
class TestStatus(Enum):
    """The test status enum."""
    PASSED = 1
    FAILED = 2
    SKIPPED = 3
    ERROR = 4

@dataclasses.dataclass
class TestResult:
    """The test result dataclass."""
    name: str
    status: TestStatus

### DO NOT MODIFY THE CODE ABOVE ###
### Implement the parsing logic below ###

def parse_test_output(stdout_content: str, stderr_content: str) ->
List[TestResult]:
    """
    Parse the test output content and extract test results.
    """
    raise NotImplementedError('Implement the test output parsing logic')
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.225Z

```python
### Implement the parsing logic above ###
### DO NOT MODIFY THE CODE BELOW ###


def export_to_json(results: List[TestResult], output_path: Path) -> None:
    json_results = {
        'tests': [
            {'name': result.name, 'status': result.status.name} for result in results
        ]
    }
    with open(output_path, 'w') as f:
        json.dump(json_results, f, indent=2)


def main(stdout_path: Path, stderr_path: Path, output_path: Path) -> None:
    with open(stdout_path) as f:
        stdout_content = f.read()
    with open(stderr_path) as f:
        stderr_content = f.read()


    results = parse_test_output(stdout_content, stderr_content)
    export_to_json(results, output_path)


if __name__ == '__main__':
    if len(sys.argv) != 4:
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.225Z

```
        print('Usage: python parsing.py <stdout_file> <stderr_file>
<output_json>')

        sys.exit(1)



    main(Path(sys.argv[1]), Path(sys.argv[2]), Path(sys.argv[3]))
```

4. **Golden solution files and test suite:** Make sure that all of the solution and test files are present, so the code can be run properly.

Execution

After making sure that all of the requirements are present and strictly following the guidelines, the environment can be run using the following commands:

Important: These commands must be run in the project root folder, where the Dockerfile is located.

1. **"docker build -t <image_name> ."** - This will be used to build a Docker image based on the Dockerfile and its instructions. Make sure you keep organized with the image names to avoid issues down the road.

   Example:

```shell
docker build -t real-coder-task-1 .
```

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.225Z

```
root@HUAN-PC:/mnt/c/dev/scale/real_coder/test_task/codebase# docker build -t real-coder-task-1 .
[+] Building 1.1s (13/13) FINISHED                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 1.91kB                                                                        0.0s
 => [internal] load metadata for docker.io/library/ubuntu:22.04                                               0.7s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [1/8] FROM docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a  0.0s
 => => resolve docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a146afb8c525a  0.0s
 => [internal] load build context                                                                             0.2s
 => => transferring context: 2.92kB                                                                           0.2s
 => CACHED [2/8] RUN apt-get update && apt-get install -y     git     python3     python3-pip     python3-setupto  0.0s
 => CACHED [3/8] RUN python3 -m venv /opt/venv                                                                0.0s
 => CACHED [4/8] WORKDIR /app                                                                                 0.0s
 => CACHED [5/8] RUN pip install --upgrade pip &&     pip install --no-cache-dir     "numpy>=1.24,<3"     "opencv  0.0s
 => CACHED [6/8] COPY . .                                                                                     0.0s
 => CACHED [7/8] RUN git init    && git config --global user.email "agent@example.com"     && git config --globa  0.0s
 => CACHED [8/8] RUN mkdir -p /eval_assets                                                                    0.0s
 => exporting to image                                                                                        0.1s
 => => exporting layers                                                                                       0.0s
 => => exporting manifest sha256:2ace9fe83c3ee789eb17ec00f5c3f0900ff4cdc1e80657b04a2898dcf0a06802             0.0s
 => => exporting config sha256:cd34da5e464c5105ea3b88700a85a6b1327f1452880d4d3ba25b5f1053dabf0f               0.0s
 => => exporting attestation manifest sha256:68d90bf3a3fb98d702a90e8afb258600a6098a9395754e6431b32a5541d938d6  0.0s
 => => exporting manifest list sha256:ff2a2cc37e8cf99ff3ded5b2aa3d7bfdb8ce86da5a4da24f2b6747d5eef6f746        0.0s
 => => naming to docker.io/library/real-coder-task-1:latest                                                   0.0s
 => => unpacking to docker.io/library/real-coder-task-1:latest                                                0.0s
root@HUAN-PC:/mnt/c/dev/scale/real_coder/test_task/codebase#
```

2. **“docker run -it <image_name>:latest /bin/bash”** - This command allows Docker to both run the container with the image created in the previous command, while also entering a shell inside of it so you can execute commands. Keep in mind that the image name used in the command must be the same created in the last step.

   Example:

```shell
docker run -it real-coder-task-1:latest /bin/bash
```

```
root@HUAN-PC:/mnt/c/dev/scale/real_coder/test_task/codebase# docker run -it real-coder-task-1:latest /bin/bash
root@8d4faf09ad2e:/app#
```

3. In the container shell, the ***run.sh*** file will be able to be executed using the **“bash run.sh”** command. If it doesn’t produce any result or ends up in an error, make sure to fix those files.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.225Z

```
root@8d4faf09ad2e:/app# bash run.sh
Running all tests...
============================= test session starts ==============================
platform linux -- Python 3.10.12, pytest-9.0.2, pluggy-1.6.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
rootdir: /app
configfile: pyproject.toml
plugins: cov-7.0.0
collecting ... collected 106 items

tests/test_clustering.py::TestClusteringContract::test_cluster_embeddings_returns_array_like PASSED [  0%]
tests/test_clustering.py::TestClusteringContract::test_cluster_labels_integer_like PASSED [  1%]
tests/test_clustering.py::TestClusteringContract::test_clustering_synthetic_dataset_expected_identities PASSED [  2%]
tests/test_clustering.py::TestClusteringIdenticalEmbeddings::test_identical_embeddings_same_cluster PASSED [  3%]
tests/test_clustering.py::TestClusteringIncremental::test_incremental_clustering_optional_interface PASSED [  4%]
tests/test_clustering.py::TestClusteringEdgeCases::test_empty_embeddings_raises PASSED [  5%]
tests/test_clustering.py::TestClusteringEdgeCases::test_none_raises PASSED [  6%]
tests/test_clustering.py::TestClusteringEdgeCases::test_wrong_shape_1d_raises PASSED [  7%]
tests/test_clustering.py::TestClusteringEdgeCases::test_large_batch_handled PASSED [  8%]
tests/test_duplicate_detection.py::TestDuplicateDetectionContract::test_build_photo_groups_empty_input_zero_photos PASSE
D [  9%]
tests/test_duplicate_detection.py::TestDuplicateDetectionContract::test_organizer_output_allows_duplicate_metadata PASSE
D [ 10%]
tests/test_edge_cases.py::TestEdgeCaseZeroFaces::test_zero_faces_returns_empty_list PASSED [ 11%]
tests/test_edge_cases.py::TestEdgeCaseSmallAndBlurry::test_extremely_small_image PASSED [ 12%]
tests/test_edge_cases.py::TestEdgeCaseSmallAndBlurry::test_low_res_face_crop_embedding PASSED [ 13%]
tests/test_edge_cases.py::TestEdgeCaseMultipleFacesPerImage::test_multiple_faces_structure PASSED [ 14%]
tests/test_edge_cases.py::TestEdgeCaseIdenticalEmbeddings::test_identical_embeddings_same_label PASSED [ 15%]
tests/test_edge_cases.py::TestEdgeCaseLargeBatch::test_large_batch_clustering PASSED [ 16%]
tests/test_edge_cases.py::TestInvalidInputsClearExceptions::test_detection_none PASSED [ 16%]
```

If you lose track of the Docker image you need to run, the **“docker images”** command can help you by listing all of the available images in your environment.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.225Z